

Projet 2 - Le backend

Application de gestion de pizzeria

1. Contexte

Pour cette 2^{ème} partie du projet de session, vous aurez à construire le “backend” de votre application web ainsi qu’à rajouter des interactions entre le “backend” et votre “frontend” existant. Votre point de départ est ce que vous avez remis en Projet 1. L’objectif de ce travail est de mettre en application les notions apprises en cours pour la partie “backend”.

2. Exigences fonctionnelles

2.1. Consigne générale

Votre application doit être structurée selon l’approche “Single Page Application”. L’ensemble des fonctionnalités doit être accessible à partir de la même page et les interactions avec le serveur doivent être faites en JavaScript.

Pour ce laboratoire, le stockage des données côté serveur se fait en mémoire seulement. Pour cela, vous pouvez utiliser l’approche suivante. Les données persisteront entre les requêtes HTML, mais ne persisteront pas après le redémarrage de l’application.

```
...
var pizzas = [];
routeur.get("/", function(req, res) {
    ...
    pizzas.forEach(...);
    pizzas.push(...);
    ...
});
...
```

2.2. Point d’entrée – 5%

Ajouter un point d’entrée à la racine (chemin d’accès “/”) qui retourne la page HTML de l’application.

2.3. API pour les opérations de gestion – 10%

a. API pour les garnitures viande

Développer un API côté serveur pour la gestion des garnitures viande. Cet API doit être construit selon les principes REST. Il doit permettre les opérations suivantes :

- Obtenir la liste des garnitures viande.
- Ajouter une variété de garnitures viande.
- Supprimer une variété de garnitures viande.

b. API pour les garnitures légumes

Développer un API côté serveur pour la gestion des garnitures légumes. Cet API doit être construit selon les principes REST. Il doit permettre les opérations suivantes :

- Obtenir la liste des garnitures légumes.
- Ajouter une variété de garnitures légumes.
- Supprimer une variété de garnitures légumes.

2.4. API pour la commande – 10%

Développer un API côté serveur pour la gestion des commandes. Cet API doit être construit selon les principes REST. Il doit permettre les opérations suivantes :

- Obtenir la liste des commandes
- Ajouter une nouvelle commande

2.5. Interaction avec le serveur (Garniture-Format) – 10%

Modifier le “frontend” existant pour que la liste des garnitures et formats affichée soit créée dynamiquement à partir des données retournées par l’API serveur (voir point 2.3).

2.6. Interaction avec le serveur (commandes) – 10%

Modifier le “frontend” existant pour que l’opération d’ajout d’une commande utilise l’opération serveur d’ajout de commande et rafraîchisse la liste des commandes à partir des données retournées par l’API serveur (voir point 2.4).

2.7. Interaction avec le serveur (opérations de gestion) – 10%

Les opérations d’ajout et de suppression ne sont pas directement accessibles à partir du “frontend”. Pour ce point, on vous demande de créer une requête vers cet API dans Postman et de fournir Le fichier json exporté.

3. Rapport

Question #1.1 – 5%

Expliquez l’approche que vous avez utilisée pour construire le HTML de la liste des garnitures/formats pizzas et des commandes.

Question #1.2 – 5%

Quels sont les avantages et les inconvénients de l'utilisation de l'API DOM ? Proposez une autre approche, notamment pour gérer des données qui changent peu.

Question #2 – 5%

Les opérations d'ajout et de retrait de garnitures ne sont pas disponibles dans l'interface du "frontend". Est-ce que cela empêche un utilisateur de faire ces opérations ? Expliquez.

Question #3.1 – 5%

Expliquez comment est-ce que vous avez utilisé des middlewares pour améliorer le fonctionnement des APIs ?

Question #3.2 – 5%

Si vous avez choisi de conserver l'ensemble du code JavaScript du "frontend" en un seul fichier, expliquez comment vous pourriez diviser le code en plusieurs fichiers pour faciliter son écriture et sa maintenance.

Si vous avez divisé le code JavaScript du "frontend" en plusieurs fichiers, expliquez ce que vous avez fait.

4. Modalités :**4.1. Remise**

Le projet est à remettre le 01/04/2026

Vous devez remettre un fichier ZIP contenant

- L'ensemble des fichiers HTML, CSS, JavaScript et configuration Docker (ou un lien GitLab ou Github).
- Fichier postman
- Un rapport au format docx ou pdf répondant aux questions de la section "Rapport", Ce rapport doit être nommé "rapport_nomEquipe" selon le format choisi.
- Une seule remise par équipe

4.2. Barème

La correction se fait selon le barème suivant :

- Exigences fonctionnelles (55%)
- Respect des bonnes pratiques HTML, CSS et JavaScript. (15%)
- Réponses aux questions (Rapport) (25%)
- Qualité du rapport (page de garde, introduction, réponses bien détaillées, conclusion), qualité du français (5%)

4.3. Consignes générales

- L'utilisation d'IA pour ce travail est interdite.
- Le Respect de la date de remise est obligatoire.